

Summary

This video explains the core methodologies for constructing Regular Expressions (RE) within the Theory of Computation, focusing on quantitative and parity-based constraints over alphabets $\{a, b\}$ or $\{0, 1\}$. A central directive for accurate RE construction is that expressions should conceptually be derived from Non-deterministic Finite Automata (NFA) rather than Deterministic Finite Automata (DFA), in order to avoid incorporating non-productive states.

Key Takeaways:

- **The Non-Productive State Rule:** Before deriving an RE from a DFA, remove all dead, unreachable, and equivalent states. This effectively reduces the DFA to an NFA that generates only valid strings.
- **Structural Flexibility:** A single language may admit multiple correct RE representations. The placement of Kleene stars (e.g., b^*) determines flexibility in accommodating unconstrained characters.
- **Parity Complexity:** Even/Odd constraints require careful handling of the unconstrained alphabet to ensure inclusion of ϵ and strings consisting solely of the unconstrained symbol.
- **Examination Trends:** Competitive examinations such as GATE frequently test “At Least” and parity-based constructions, particularly identifying redundant or insufficient placements of b^* .

1. Foundational Methodology: From Automata to Regular Expressions

The construction of a Regular Expression is intrinsically connected to Finite Automata. However, a distinction must be made between DFA and NFA during derivation.

1.1 The Role of Non-Productive States

A DFA is a complete system that includes transitions for every symbol in the alphabet from every state. This often introduces dead states for invalid strings.

Rule: When deriving a Regular Expression, remove non-productive states (dead states, unreachable states, and equivalent states).

Rationale: Regular Expressions, like NFAs, generate only valid strings. Dead states do not contribute to the language and unnecessarily complicate the structure.

Result: Removing such states effectively transforms the DFA into an NFA, providing a cleaner path toward expression construction.

1.2 Quantitative Constraint Patterns

For constraints involving exact counts of a symbol (e.g., exactly n occurrences of a), the unconstrained symbol (b) must be allowed in all valid positions using b^* .

Constraint Type	Logic for Number of a's over $\Sigma = \{a, b\}$	Example Regular Expression
Exactly 1	One 'a' with any number of 'b's before or after	$b^* a b^*$
Exactly 2	Two 'a's with 'b's allowed in all three gaps	$b^* a b^* a b^*$
At Most 1	Either zero 'a's (only b^*) or exactly one 'a'	$b^* + b^* a b^*$
At Most 2	Zero, one, or two 'a's	$b^* + b^* a b^* + b^* a b^* a b^*$

2.

Advanced Quantitative Constraints: "At Least" Logic

"At Least" constraints are more complex because they allow infinite repetition of the constrained symbol after the minimum requirement is satisfied.

2.1 At Least 1 a

The expression must ensure at least one occurrence of 'a', while allowing any combination of symbols elsewhere.

Valid constructions:

- $b^* a (a+b)^*$
- $(a+b)^* a (a+b)^*$
- $(b^* a b^*)^+$

The last form ensures at least one repetition of the "Exactly 1" pattern.

2.2 At Least 2 a's

Two 'a's must appear as a base, with flexibility around and between them.

Primary form:

$b^* a b^* a (a+b)^*$

Distributed form:

$(a+b)^* a (a+b)^* a (a+b)^*$

Common Error:

$(b^* a b^* a b^*)^+$

This generates only even counts (2, 4, 6, ...) and fails to generate strings with exactly 3 a's. Therefore, it does not represent "At Least 2."

3. Parity and Modulo Constraints

Parity-based languages are conceptually subtle due to the interaction between constrained and unconstrained symbols.

3.1 Even Number of a's

The core idea is repetition of the “Exactly 2” structure.

Base form:

$(b^* a b^* a b^*)^*$

This generates 0, 2, 4, ... occurrences of a.

Handling Zero a's:

The expression must also accept strings consisting only of b's, including ϵ .

Redundancy and Necessity of b^* :

- At least one external b^* must allow generation of strings with no a.
- The internal b^* between the two a's is mandatory to allow b's between them.
- Some internal b^* may become redundant depending on star placement.

3.2 Odd Number of a's

An odd count can be constructed as:

Even count followed by one a:

$(b^* a b^* a b^*)^* (b^* a b^*)$

Or:

One a followed by an even count:

$(b^* a b^*) (b^* a b^* a b^*)^*$

Both constructions ensure 1, 3, 5, ... occurrences of a.

3.3 0 mod 3 Constraints

To generate multiples of three a's:

$(b^* a b^* a b^* a b^*)^*$

As with even parity, correct placement of b^* ensures acceptance of strings with zero a's (e.g., bb).

4. Analytical Case Studies: GATE Examination Patterns

4.1 GATE 2008: Exactly Two 1s

Question: Identify the RE for exactly two 1s over $\Sigma = \{0, 1\}$.

Correct expression:

$0^* 1 0^* 1 0^*$

Insight: The structure mirrors the “Exactly 2” pattern with 0^* replacing b^* .

4.2 GATE 2009: At Least Two 0s

Expression:

$(0+1)^* 0 (0+1)^* 0 (0+1)^*$

Insight: The two fixed 0's ensure the minimum requirement. The surrounding $(0+1)^*$ segments act as placeholders for arbitrary strings.

Language: Strings with at least two 0s.

4.3 GATE 2010: Odd Number of 1s (Marks to All)

The question was declared “Marks to All” because all options failed to generate the simplest valid odd string (e.g., 010). Many incorrect options implicitly enforced a minimum of two or more 1s due to flawed Kleene star placement.

4.4 GATE 2024: Intersecting Constraints

Problem: Count strings of length ≤ 4 having an odd number of a's.

Solution:

Total strings up to length 4:

$$\begin{aligned} &\sum \text{from } i=0 \text{ to } 4 \text{ of } 2^i \\ &= 1 + 2 + 4 + 8 + 16 \\ &= 31 \end{aligned}$$

For any length $n > 0$, exactly half the strings have odd a's.

Breakdown:

Length 1: 2 strings $\rightarrow$ 1 odd

Length 2: 4 strings $\rightarrow$ 2 odd

Length 3: 8 strings $\rightarrow$ 4 odd

Length 4: 16 strings $\rightarrow$ 8 odd

Total odd strings:

$$1 + 2 + 4 + 8 = 15$$

